


```
1391: # setup for the invertible network
DEFAULT_SETTINGS_DICT = {
    "dense_args": dict(layers=2, units=128, activation=nn.ReLU(),
        "dropout": True,
        "batch_norm": 0.05,
        "dropout_prob": 0.05,
        "soft_clamping": 1
    )
}

# testing scaling and shifting network (Conditioned) for the coupling blocks
class DenseNet(nn.Module):
    def __init__(self, dim_in, dim_out, dim_cond, settings, **kwargs):
        super().__init__(**kwargs)

        try:
            activation = settings['activation']
            layers = settings['layers']
            units = settings['units']
        except:
            raise ValueError("Please provide the settings for the dense network")

        self.net = nn.ModuleList()
        for l in range(layers):
            self.net.add_module(f"Linear_{l}", nn.Linear(dim_in + dim_cond if l == 0 else units, units))
            self.net.add_module(f"Activation_{l}", nn.ReLU())
            if settings.get("batch_norm", False):
                self.net.add_module(f"BatchNorm_{l}", nn.BatchNorm1d(units))
            if settings.get("dropout", False):
                self.net.add_module(f"Dropout_{l}", nn.Dropout(settings.get("dropout_prob", 0.05)))
            self.net.add_module(f"Linear_out", nn.Linear(units, dim_out))

    def forward(self, input, condition):
        x = torch.cat([input, condition], dim=-1)
        for layer in self.net:
            x = layer(x)
        return x

# Affine Coupling Layer
class AffineCoupling(nn.Module):
    def __init__(self, dim_split_1, dim_split_2, coupling_settings, **kwargs):
        super().__init__(**kwargs)

        dense_settings = coupling_settings.get('dense_args', {})
        assert dense_settings != {}, "Please provide dense settings for the coupling layer"
        assert coupling_settings.get('n_summary_stats') is not None, "Please provide the dimension of the condition as a Gaussian latent space as a source distribution"
        self.soft_clamp = coupling_settings.get('soft_clamping', None)

        self.scale = DenseNet(dim_split_1, dim_split_2, coupling_settings['n_summary_stats'], dense_settings)
        self.translate = DenseNet(dim_split_1, dim_split_2, coupling_settings['n_summary_stats'], dense_settings)

    def forward(self, split1, split2, condition, inverse=False, **kwargs):
        if not inverse:
            return self._forward(split1, split2, condition, **kwargs)
        else:
            return self._inverse(split1, split2, condition, **kwargs)

    def _forward(self, u1, u2, condition, **kwargs):
        s = self.scale(u2, condition, **kwargs)
        if self.soft_clamp is not None:
            s = (2.0 * self.soft_clamp / torch.pi) * torch.atan(s / self.soft_clamp)
        v = u1 * torch.exp(s) + t
        log_det = torch.sum(s, dim=-1)
        return v, log_det

    def _inverse(self, v1, v2, condition, **kwargs):
        s = self.scale(v1, condition, **kwargs)
        if self.soft_clamp is not None:
            s = (2.0 * self.soft_clamp / torch.pi) * torch.atan(s / self.soft_clamp)
        t = self.translate(v1, condition, **kwargs)
        u = (v2 - t) * torch.exp(-s)
        return u

class CouplingLayer(nn.Module):
    def __init__(self, latent_dim, dim_rot, coupling_settings: dict = {}, **kwargs):
        super().__init__(**kwargs)

        self.latent_dim = latent_dim
        self.dim_out1 = latent_dim // 2
        self.dim_out2 = latent_dim - self.dim_out1
        merged_coupling_settings = DEFAULT_SETTINGS_DICT_COUPLING
        for key, value in coupling_settings.items():
            merged_coupling_settings[key] = value

        # The dim-one coupling block (i.e., no inactive part after a forward pass)
        self.net1 = AffineCoupling(self.dim_out1, self.dim_out2, coupling_settings=merged_coupling_settings)
        self.net2 = AffineCoupling(self.dim_out1, self.dim_out2, coupling_settings=merged_coupling_settings)

        self.rotation = torch.tensor(special_ortho_group_rvs(latent_dim) if has_rotation else torch.eye(latent_dim), dtype=torch.float32)

    def forward(self, inputs, condition, inverse=False, **kwargs):
        if not inverse:
            return self._forward(inputs, condition, **kwargs)
        else:
            return self._inverse(inputs, condition, **kwargs)

    def _forward(self, inputs, condition, **kwargs):
        log_det_tot = torch.zeros(1)
        inputs = inputs @ self.rotation.T
        return inputs

    # Pass through coupling layer
    def _forward(self, u1, u2, condition, **kwargs):
        v1, log_det1 = self.net1(u1, u2, condition, **kwargs)
        v2, log_det2 = self.net2(u2, v1, condition, **kwargs)
        latent, log_det = torch.cat([v1, v2], dim=-1), log_det1 + log_det2
        return latent, log_det

    def _inverse(self, latent, condition, **kwargs):
        v1, v2 = torch.split(latent, (self.dim_out1, self.dim_out2), dim=-1)
        u1 = self.net1(v1, v2, condition, inverse=True, **kwargs)
        u2 = self.net2(u2, v1, condition, inverse=True, **kwargs)
        inputs = torch.cat([u1, u2], dim=-1)
        return inputs

    inputs = inputs @ self.rotation.T
    return inputs

# Conditional Invertible neural network with affine or RealNVP coupling layers
class ConditionalRealNVP(nn.Module):
    def __init__(self, n_params, n_coupling_layers, coupling_settings={}, **kwargs):
        super().__init__(**kwargs)

        layer_settings = dict(
            latent_dim=latent_dim,
            dim_rot=latent_dim,
            coupling_settings=dict = {},
            **kwargs
        )

        self.coupling_layers = self._create_coupling_layers(n_coupling_layers, layer_settings, coupling_settings)

    def _create_coupling_layers(self, n_coupling_layers, layer_settings, coupling_settings):
        return nn.ModuleList([CouplingLayer(*layer_settings, has_rotation=kl=0, coupling_settings=coupling_settings) for i in range(n_coupling_layers)])
```

```
In [28]: # loss functions
def nll_loss_gaussian(z, log_det_0):
    """
    Computes the Kullback-Leibler divergence between the true and approximate
    posterior assuming a Gaussian latent space as a source distribution.
    """
    loss = torch.mean(0.5 * torch.square(torch.norm(z, dim=-1) - log_det_0))
    return loss

def kernel(X, Y, sigma, reduce_mean=True):
    mat = X[:,None]-Y
    mat = mat**2
    mat = -mat/(float(sigma)**2) * 2)
    mat = torch.exp(mat)
    if reduce_mean:
        mat = torch.mean(mat)
    return mat

def mmd(X, Y, sigma=(0.1, 0.5, 1, 2, 5)):
    if isinstance(sigma, (int, float)):
        return torch.sqrt(torch.abs(kernel(X, X, sigma) + kernel(Y, Y, sigma) - 2 * kernel(X, Y, sigma)))
    elif isinstance(sigma, list):
        total_mmd = 0
        for kernel_size in sigma:
            total_mmd += mmd(X, Y, kernel_size)
        return total_mmd

We turn the point estimator network into a summary network by dropping the last layer.
```

```
In [33]: # training loop for inn
def train_inn(data_config, summary_config, train_config, inn_config):
    prior_simulator = data_config['prior'], data_config['simulator']
    data_samples = {'prior': prior_simulator, 'observations': simulator(prior_simulator)}
    train_loader = create_data_loader(data_config['n_samples'], prior_simulator, batch_size=train_config['batch_size'], shuffle=train_config['shuffle'])
    validation_loader = create_data_loader(data_config['n_validation'], prior_simulator, batch_size=train_config['batch_size'], shuffle=train_config['shuffle'])

    inference_network = ConditionalRealNVP(data_samples['prior'].shape[-1], n_coupling_layers=inn_config['n_coupling_layers'], coupling_settings=inn_config['coupling_settings'])
    summary_network = SummaryNN(feature_dim=data_samples['observations'].shape[-1], n_params_to_predict=data_samples['prior'].shape[-1], last_layer=False, **summary_config)

    optimizer = torch.optim.Adam(list(inference_network.parameters()) + list(summary_network.parameters()))
    lr_scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, 'min')

    loss_history = {'train': [], 'validation': []}
    epochs = int(train_config['training_steps'] / len(train_loader))
    with tqdm(range(epochs)) as bar:
        for i in range(epochs):
            for l in range(1, len(summary_network.parameters())):
                summary_network.train(True)
                inference_network.train(True)
                optimizer.zero_grad()

                sim_params = sim.to(train_config['device']), params_to(train_config['device'])
                summary_with_noise = params + torch.randn_like(params) * train_config['added_noise']
                summary, log_det = summary_network(summary_with_noise)
                z, log_jacobians = inference_network(summary_with_noise, summary_stats)
                vloss = nll_loss_gaussian(z, log_jacobians)
                if train_config.get('mmd_factor', 0.0) > 0.0:
                    vloss += mmd(summary_stats, torch.randn_like(summary_stats)) * train_config['mmd_factor']

                loss_history['train'].append(loss.item())

            loss_backward()

            # Gradient clipping to prevent exploding gradients and unstable training
            nn.utils.clip_grad_norm_(list(inference_network.parameters()) + list(summary_network.parameters()), max_norm=train_config['gradient_clipping'])
            optimizer.step()

            # include validation loss to detect overfitting
            inference_network.eval()
            with torch.no_grad():
                for i in range(1, len(summary_network.parameters())):
                    summary_network.train(True)
                    inference_network.train(True)
                    summary_with_noise = summary_network(summary_with_noise)
                    z, log_jacobians = inference_network(summary_with_noise, summary_stats)
                    vloss = nll_loss_gaussian(z, log_jacobians)
                    if train_config.get('mmd_factor', 0.0) > 0.0:
                        vloss += mmd(summary_stats, torch.randn_like(summary_stats)) * train_config['mmd_factor']

                    loss_history['validation'].append(vloss.item())
                    bar.set_postfix(loss=loss.item(), val_loss=vloss.item())

    return loss_history, summary_network, inference_network
```

```
In [34]: # network and training configs
train_config = {
    'batch_size': 512,
    'shuffle': True,
    'training_steps': 10000, # instead of fixed epochs - to have the same nr of steps when varying n_samples & batch_size
    'learning_rate': 0.0001,
    'gradient_clipping': 1.0,
    'weight_decay': 1e-4,
    'added_noise': 0.0, # code is already prepared for exercise 4b, where we will add noise
    'loss_fn': nll_loss_gaussian,
    'mmd_factor': 0.0, # by default, we impose no MMD regularization on the summary statistics
    'device': "cuda" if torch.cuda.is_available() else "cpu"
}

summary_config = {
    'conv_channel_sizes': [1],
    'dense_sizes': [512, 512],
    'dropout': 0.0,
    'batch_norm': False,
    'activation': nn.ReLU(),
}

inn_config = dict({
    'n_coupling_layers': 8,
    'coupling_settings': {
        'dense_args': summary_config['dense_sizes'][-1],
        'dense_args': dict(layers=2, units=128, activation=nn.ReLU(),
            "dropout": False,
            "dropout_prob": 0.05
        )
    }
})

data_config = dict({
    'n_samples': 10000,
    'n_validation': 100,
    'prior': prior,
    'simulator': simulator,
    'n_test': 1000,
    'n_posterior_samples': 100
})
```

By adding an INN, we aim to learn the uncertainty in the parameters. For an inverse problem, there can be various sources of uncertainty: It will obviously arise when the simulation is stochastic, but we restrict ourselves to a deterministic simulation for now. Even then, the information loss in the mapping from parameters to observables can lead to parameter ambiguity. However, we observed in the point estimation experiments above that a regression network captures the parameters very well. Thus, we expect the uncertainty due to information loss and ambiguity to be small.

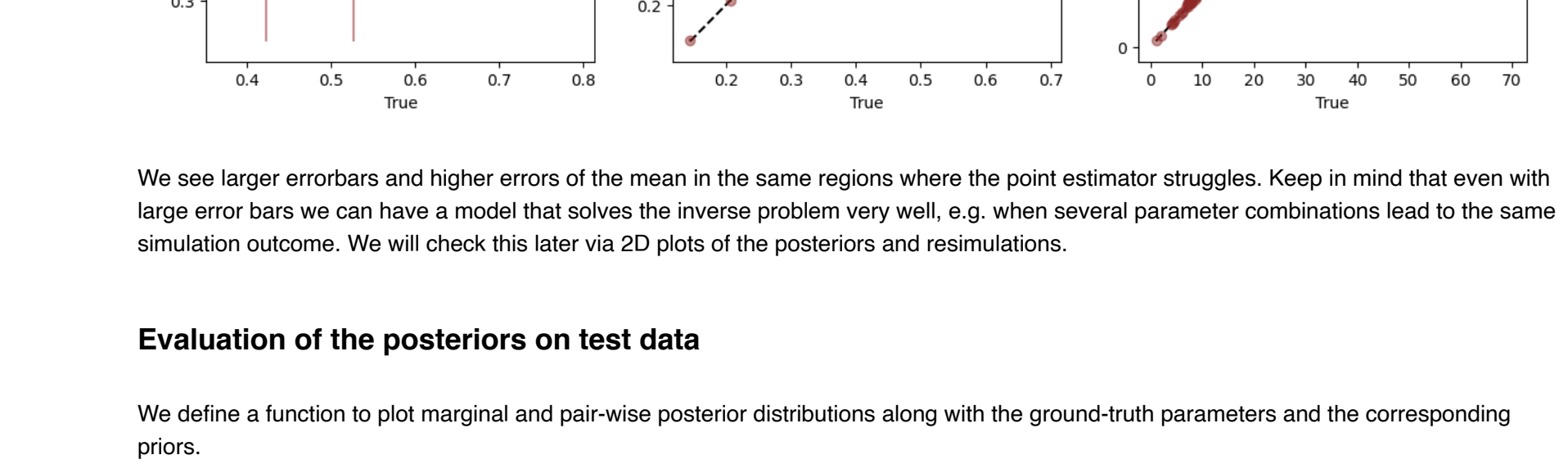
```
In [40]: loss_history, summary_net, inn = train_inn(data_config, summary_config, train_config, inn_config)

C:\Users\ukoethe\AppData\Local\Temp\ipykernel_47088\1223895618.py:95: UserWarning: To copy construct from a tensor, it is recommended to use sourceTensor.detach().clone() or sourceTensor.detach().clone().requires_grad_(True), rather than torch.tensor(sourceTensor).
  self.rotation = torch.tensor(special_ortho_group_rvs(latent_dim) if has_rotation else torch.eye(latent_dim), dtype=torch.float32)
100%|██████████| 500/500 [38:00<00:00, 4.56s/it, loss=-11.1, val_loss=-10.2]
```

```
In [41]: validation_point = np.linspace(0, len(loss_history['train']), len(loss_history['validation']))

plt.figure(figsize=(10, 5))
plt.plot(loss_history['train'], label='Train')
plt.plot(validation_point, loss_history['validation'], label='Validation')

plt.ylabel('Loss')
plt.xlabel('Train step')
plt.title('Loss history with training data from deterministic simulation')
plt.ylim([-15, 4])
plt.legend()
plt.show()
```



We again generate a test set of 1000 simulations from the prior and run the trained INN and summary network on it.

```
In [73]: test_data = get_test_data(summary_net, inn, data_config=data_config, noise=0.0)

C:\Users\ukoethe\AppData\Local\Temp\ipykernel_47088\1223895618.py:95: UserWarning: To copy construct from a tensor, it is recommended to use sourceTensor.detach().clone() or sourceTensor.detach().clone().requires_grad_(True), rather than torch.tensor(sourceTensor).
  self.rotation = torch.tensor(special_ortho_group_rvs(latent_dim) if has_rotation else torch.eye(latent_dim), dtype=torch.float32)
100%|██████████| 500/500 [38:00<00:00, 4.56s/it, loss=-11.1, val_loss=-10.2]
```

To compare the INN-based posterior estimates to the point estimates from above, we calculate the mean and 90% confidence bands from the predicted posteriors and plot them against the ground-truth values.

```
In [112]: plot_color = "#8f2727"

def plot_recovery_distribution(prior_samples, posterior_samples, labels=None, agg_func=np.median):
    """
    Calculate the mean and 90% confidence intervals of the posterior samples
    estimates = agg_func(posterior_samples, axis=1)
    percentile_5 = np.percentile(posterior_samples, 5, axis=1)
    percentile_95 = np.percentile(posterior_samples, 95, axis=1)
    n_prior_params = prior_samples.shape[1]
    if labels is None:
        labels = [f'Parameter {i+1}' for i in range(n_prior_params)]

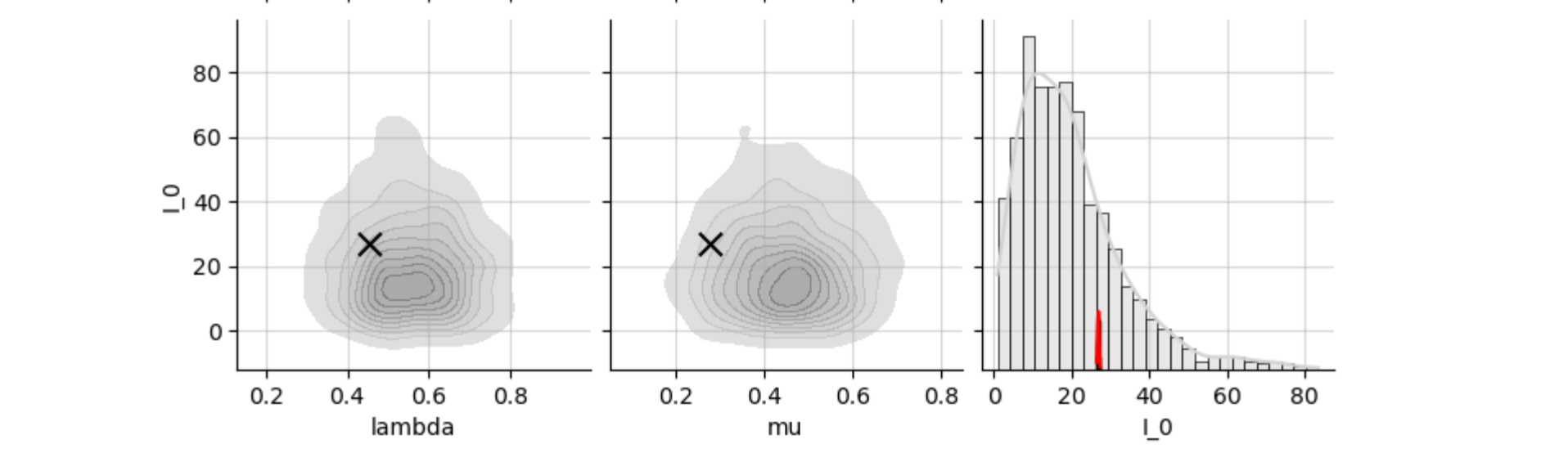
    fig, axes = plt.subplots(1, n_prior_params, figsize=(5 * n_prior_params, 5))
    if n_prior_params == 1:
        axes = [axes] # Ensure axes is iterable for a single parameter

    for i in range(n_prior_params):
        ax = axes[i] if n_prior_params > 1 else axes
        # Error bars represent the 90% CI
        yerr = [estimates[i], i] - percentile_5[i], i], percentile_95[i], i] - estimates[i], i]
        ax.errorbar(prior_samples[:, i], estimates[i], i], yerr=yerr, fmt='o', color=plot_color, alpha=0.5)
        ax.plot([prior_samples[:, i].min(), prior_samples[:, i].max()],
            [prior_samples[:, i].min(), prior_samples[:, i].max()],
            '-', color='black', label='Ideal Prediction')
        r2 = r2_score(prior_samples[:, i], estimates[i])
        ax.text(0.05, 0.95, f'R2: {r2:.2f}', transform=ax.transAxes, fontsize=12,
            verticalalignment='top')
        ax.set_title(f'{labels[i]}')
        ax.set_xlabel('True')
        ax.set_ylabel('Predicted')

    return fig
```

```
In [113]: fig = plot_recovery_distribution(test_data['prior'][:100,:], test_data['posterior'][:100,:], labels=prior.get_labels())

lambda mu I0
```



We see larger errorbars and higher errors of the mean in the same regions where the point estimator struggles. Keep in mind that even with large error bars we can have a model that solves the inverse problem very well, e.g. when several parameter combinations lead to the same outcome. We will check this later via 2D plots of the posteriors and residuals.

Evaluation of the posteriors on test data

We define a function to plot marginal and pair-wise posterior distributions along with the ground-truth parameters and the corresponding priors.

```
In [117]: def plot_recovery_distribution(ground_truth, posterior_estimates, prior_draws, labels):
    n_params = len(labels)
    df_posterior = pd.DataFrame(posterior_estimates, columns=labels)
    df_prior = pd.DataFrame(prior_draws, columns=labels)

    g = sns.PairGrid(df_posterior)
    g.map_diag(sns.histplot, fill=True, color='red', alpha=0.5, kde=True)
    g.map_lower(sns.kdeplot, fill=True, color='red', alpha=0.5)

    prior_draws_df = pd.DataFrame(prior_draws, columns=labels)
    g_diag = prior_draws_df
    g.map_diag(sns.histplot, fill=True, color='lightgrey', alpha=0.5, kde=True, zorder=-1)
    g.map_lower(sns.kdeplot, fill=True, color='lightgrey', alpha=0.5, zorder=-1)

    for i, label in enumerate(labels):
        g.axes[i, 0].set_ylabel(label)
        g.axes[i, i].set_xlabel(label)

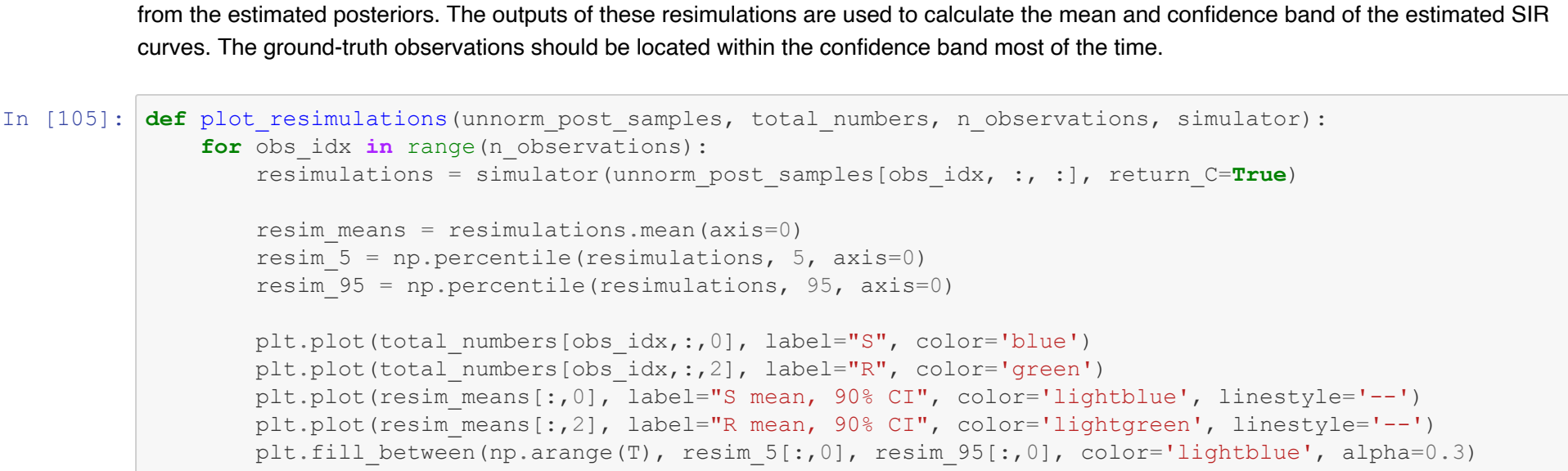
    # Add grids background
    for i in range(n_params):
        for j in range(n_params):
            g.axes[i, j].grid(alpha=0.5)
            if i == j and j < 0:
                g.axes[i, j].scatter(ground_truth[j], ground_truth[i], marker='x', color='black', s=100, zorder=10)

    for i, j in zip(range(1, n_params), range(0, n_params-1)):
        g.axes[i, j].axis("off")

    g.tight_layout()
    return g.figure
```

```
In [118]: test_index = 3
fig = plot_recovery_distribution(test_data['prior'][test_index,...], test_data['posterior'][test_index,...], test_data['prior'], prior_get_labels())

lambda mu I0
```

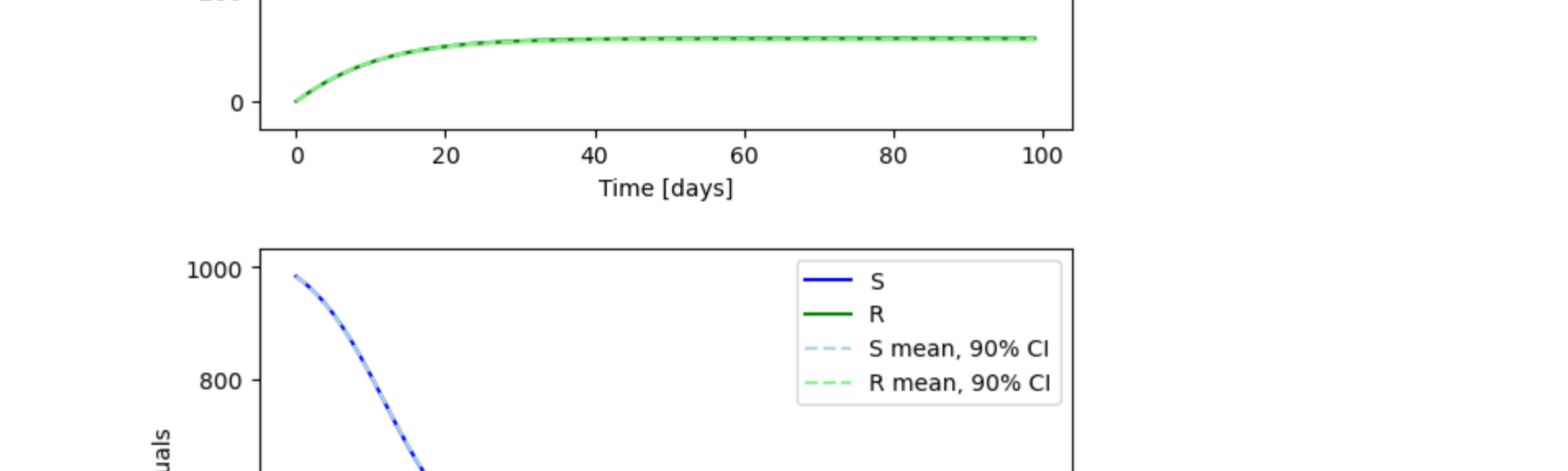


The ground truths for test instance 3 are recovered very accurately: in comparison to the prior, the posterior is extremely narrow around the ground truth values.

The results for test instance 0 differ in an interesting way:

```
In [119]: test_index = 0
fig = plot_recovery_distribution(test_data['prior'][test_index,...], test_data['posterior'][test_index,...], test_data['prior'], prior_get_labels())

lambda mu I0
```



For test instance 0, we see a high correlation between λ and μ . Higher values of λ can be compensated by higher μ to still lead to the same simulation outcome. This is a typical example of an ill-posed inverse problem, where multiple parameter combinations lead to the same observations. The INN is very useful here, since it clearly indicates the ambiguity: It returns a posterior that is elongated along the curve of compatible parameter tuples, but narrow in the other directions. In contrast, point estimators may pick an arbitrary point along the correlation curve.

To check this hypothesis, we create resimulations for the first few test instances, i.e. we run the simulation again with parameters sampled from the estimated posteriors. The outputs of these resimulations are used to calculate the mean and confidence band of the estimated SIR curves. The ground-truth observations should be located within the confidence band most of the time.

```
In [105]: def plot_resimulations(unknown_post_samples, total_numbers, n_observations, simulator):
    for obs_idx in range(n_observations):
        resimulations = simulator(unknown_post_samples[obs_idx, :, :], return_C=True)

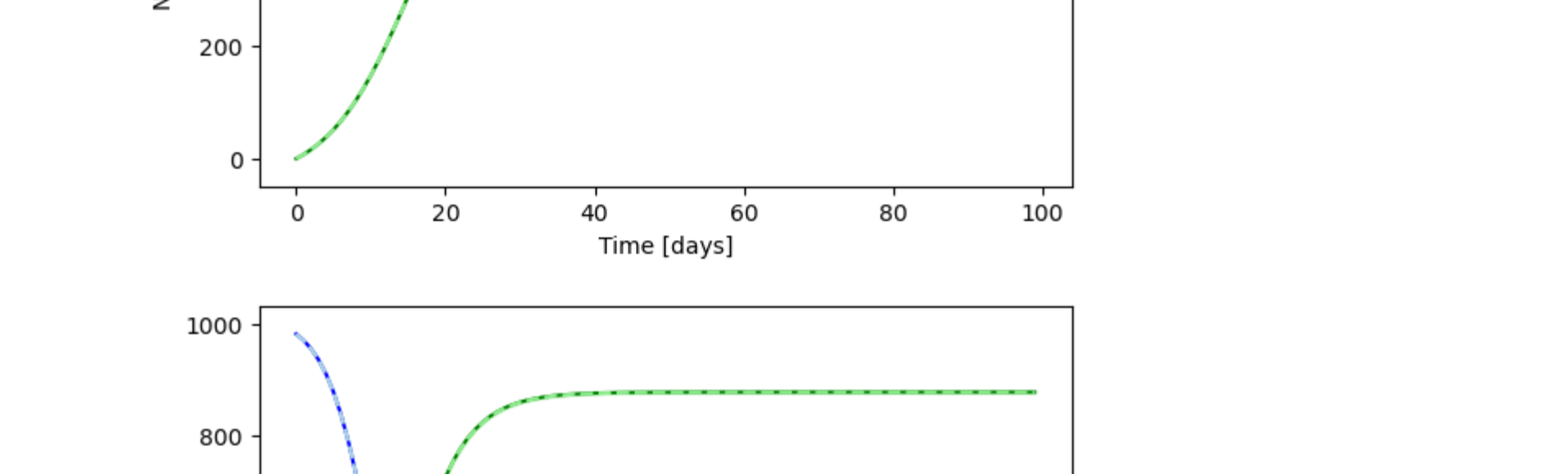
        resim_means = resimulations.mean(axis=0)
        resim_5 = np.percentile(resimulations, 5, axis=0)
        resim_95 = np.percentile(resimulations, 95, axis=0)

        plt.plot(total_numbers[obs_idx, 0], label="S", color="blue")
        plt.plot(total_numbers[obs_idx, 2], label="R", color="green")
        plt.plot(resim_means[0], label="S mean, 90% CI", color="lightblue", linestyle="--")
        plt.plot(resim_means[2], label="R mean, 90% CI", color="lightgreen", linestyle="--")
        plt.fill_between(resim_5[0], resim_95[0], color="lightblue", alpha=0.3)
        plt.fill_between(resim_5[2], resim_95[2], color="lightgreen", alpha=0.3)

        plt.xlabel("Time (days)")
        plt.ylabel("Number of individuals")
        plt.legend()
        plt.show()
```

```
In [106]: n_observations = 5
n_resimulations = 100

plot_resimulations(test_data['posterior'], test_data['C'], n_observations, data_config['simulator'])
```



The confidence bands are very narrow and cover the ground-truth curves perfectly. (It's actually a bit too perfect, and one should verify that the validation code does not contain a bug.)

4 Model misspecification detection

For model misspecification detection, we'll add an MMD loss to the latent space of the summary network that enforces a gaussian distribution. The extended loss then reads

$$\text{loss} = \text{loss}_{\text{data}} + \text{loss}_{\text{mmd}} + \lambda_{\text{mmd}}$$

The factor balances both loss terms and can be set in the train_config as mmd_factor.

```
In [138]: # network and training configs
summary_config = dict({
    'conv_channel_sizes': [1],
    'dense_sizes': [256, 50, 50],
    'dropout': 0.05,
    'batch_norm': False,
    'activation': nn.ReLU(),
})

inn_config = dict({
    'n_coupling_layers': 8,
    'coupling_settings': {
        'dense_args': summary_config['dense_sizes'][-1],
        'dense_args': dict(layers=2, units=128, activation=nn.ReLU(),
            "dropout": False,
            "dropout_prob": 0.05
        )
    }
})

data_config = dict({
    'batch_size': 1024,
    'shuffle': True,
    'training_steps': 10000,
    'learning_rate': 0.0005,
    'gradient_clipping': 1.0,
    'weight_decay': 1e-4,
    'added_noise': 0.0,
    'mmd_factor': 10.0, # switch on the MMD loss for summary network training
    'device': "cuda" if torch.cuda.is_available() else "cpu"
})

train_config = dict({
    'batch_size': 1024,
    'shuffle': True,
    'training_steps': 10000,
    'learning_rate': 0.0005,
    'gradient_clipping': 1.0,
    'weight_decay': 1e-4,
    'added_noise': 0.0,
    'mmd_factor': 10.0, # switch on the MMD loss for summary network training
    'device': "cuda" if torch.cuda.is_available() else "cpu"
})
```

```
In [139]: loss_histories_mmd, summary_network_mmd, inference_network_mmd = train_inn(data_config, summary_config,
    train_config, inn_config)

C:\Users\ukoethe\AppData\Local\Temp\ipykernel_47088\1223895618.py:95: UserWarning: To copy construct from a tensor, it is recommended to use sourceTensor.detach().clone() or sourceTensor.detach().clone().requires_grad_(True), rather than torch.tensor(sourceTensor).
  self.rotation = torch.tensor(special_ortho_group_rvs(latent_dim) if has_rotation else torch.eye(latent_dim), dtype=torch.float32)
100%|██████████| 1000/1000 [2:57:09<00:00, 10.63s/it, loss=-4.49, val_loss=-1.74]
```



```
In [140]: # plot loss histo
fig, ax = plt.subplots(1, 1, figsize=(15, 5))
ax.plot(loss_histories_mmd['train'], label='Train')
validation_point = np.linspace(0, len(loss_histories_mmd['train']), len(loss_histories_mmd['validation']
))
ax.plot(validation_point, loss_histories_mmd['validation'], label='Validation')
ax.set_ylabel('Loss')
ax.set_xlabel('Train step')
ax.legend()
```

Out[140]: <matplotlib.legend.Legend at 0x2234d46b340>

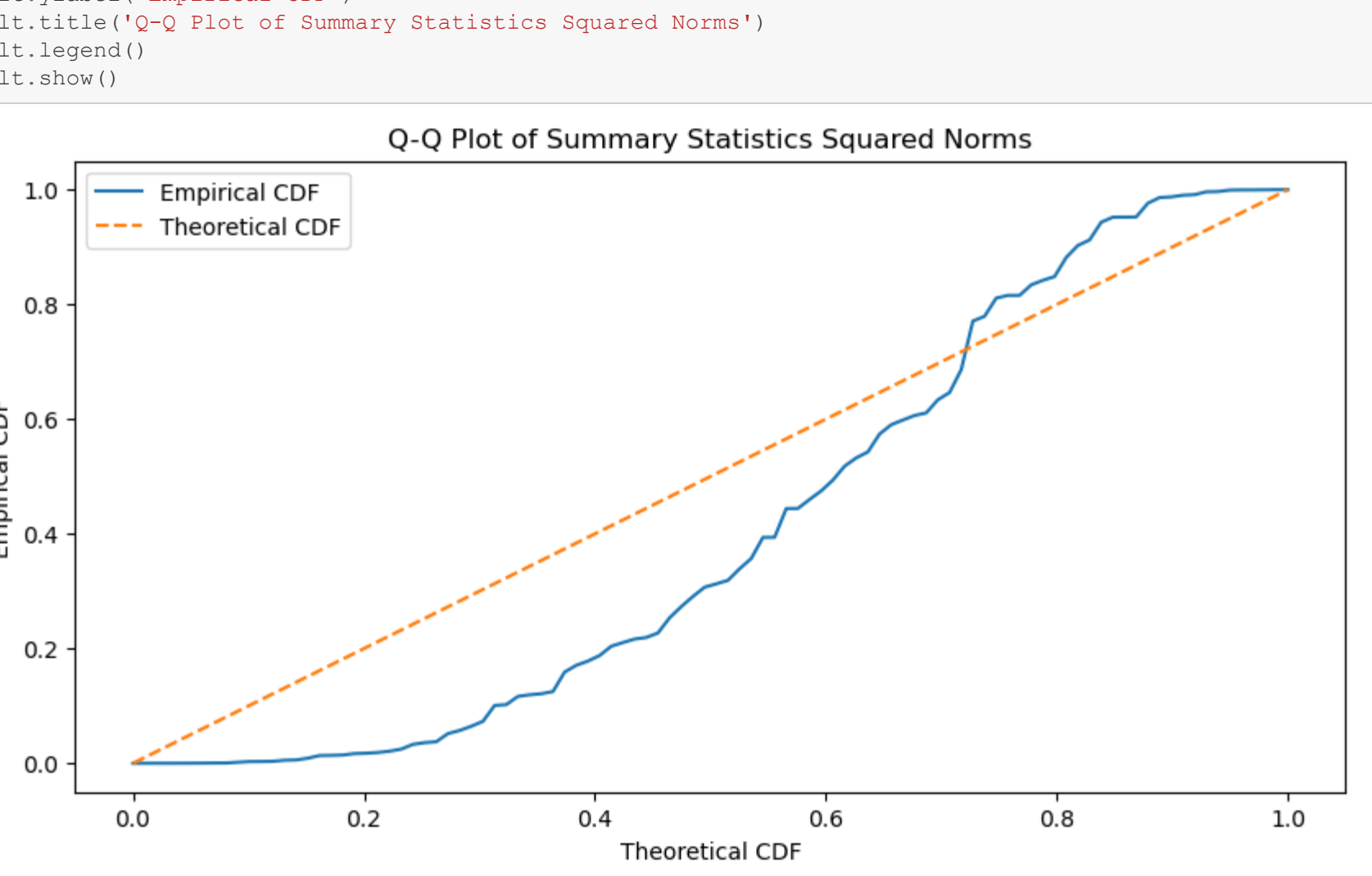


We again create test data and evaluate it with the new model.

```
In [141]: test_mmd = get_test_data(summary_network_mmd, inference_network_mmd, data_config)

C:\Users\ukoethe\AppData\Local\Temp\ipykernel_47088\3672741692.py:50: UserWarning: To copy construct from a tensor, it is recommended to use sourceTensor.detach().clone() or sourceTensor.detach().clone().requires_grad_(True), rather than torch.tensor(sourceTensor).
  post_samples[:,i,:]= inn(torch.tensor(z, dtype=torch.float32), conditions, inverse=True).detach().numpy()

In [142]: fig = plot_recovery_distribution(test_mmd['prior'][:100,:], test_mmd['posterior'][:100,:], labels=prior.get_labels())
```



Since we did not perform proper hyperparameter tuning, the results are considerably worse than before. However, the model still predicts the parameter means reasonably well, whereas the standard deviations of the posteriors became much higher.

Check if the inlier data follow a standard normal distribution in summary space, using a chi squared test and a Kolmogorov-Smirnov test.

```
In [147]: summary_network_mmd.eval()
test_prior, test_sim = generate_data(100, data_config['prior'], data_config['simulator'])
summary_inlier = summary_network_mmd(torch.tensor(test_sim, dtype=torch.float32))

s2 = torch.sum(summary_inlier**2, axis=1).detach().numpy() # and their squared norm
df = summary_inlier.shape[-1] # calculate degrees of freedom
p_values = chi2.sf(s2, df) # chi-square test
print(np.sum(p_values < 0.01), "out of", len(p_values), "samples rejected the null hypothesis at 1% level (simple chi2 test)")

stat, ks_p_value = kstest(s2, chi2(df).cdf) # Kolmogorov-Smirnov test
print("The KS test", "rejected" if (ks_p_value < 0.01) else "accepted", "the null hypothesis at 1% level with p-value", ks_p_value)

10 out of 100 samples rejected the null hypothesis at 1% level (simple chi2 test)
The KS test rejected the null hypothesis at 1% level with p-value 9.047838615139777e-06

The empirical CDF of the inlier data does indeed deviate substantially from the expected uniform CDF. Thus, the summary distribution of the inliers is still not close enough to a standard normal distribution to pass the KS test.
```

```
In [150]: # plot the first two dimensions of the summary statistics
plt.figure(figsize=(6, 6))
plt.scatter(summary_inlier[:, 0].detach().numpy(), summary_inlier[:, 1].detach().numpy(), alpha=0.5)
plt.xlabel("Summary Statistic Dimension 1")
plt.ylabel("Summary Statistic Dimension 2")
plt.title("Scatter plot of the first two dimensions of the summary statistics")
plt.grid(True)
plt.axis('equal')
plt.show()
```



This also becomes readily visible in a latent space scatter plot. Thus, we need more training and better hyperparameter tuning to get good results.

```
In [145]: # plot the first two dimensions of the summary statistics
plt.figure(figsize=(6, 6))
plt.scatter(summary_inlier[:, 0].detach().numpy(), summary_inlier[:, 1].detach().numpy(), alpha=0.5)
plt.xlabel("Summary Statistic Dimension 1")
plt.ylabel("Summary Statistic Dimension 2")
plt.title("Scatter plot of the first two dimensions of the summary statistics")
plt.grid(True)
plt.axis('equal')
plt.show()
```